

LiteLLM Chat Backend — Technical Documentation

Field	Detail
Project	LiteLLM Chat API Backend
Organization	Truviq (Internship Deliverable)
Author	Vishwas Chakilam
Version	1.0.0
Date	June 2026
Repository	github.com/Vishwas-Chakilam/LiteLLM-Backend
Live Deployment	https://litellm-chat-api.onrender.com

1. Executive Summary

This document describes the **LiteLLM Chat Backend**, a FastAPI-based REST service developed during my internship at **Truviq**. The system provides a unified chat interface over multiple LLM providers (Groq, Google Gemini) using the **LiteLLM Python SDK**, with conversation memory, intelligent model routing, cost controls, and a hardened system prompt for safer responses.

The API is designed to support **red-team security testing** with **Microsoft PyRIT**, enabling evaluators to run jailbreak and adversarial prompt campaigns against a production-like HTTP endpoint while preserving full conversation history per session.

Key outcomes:

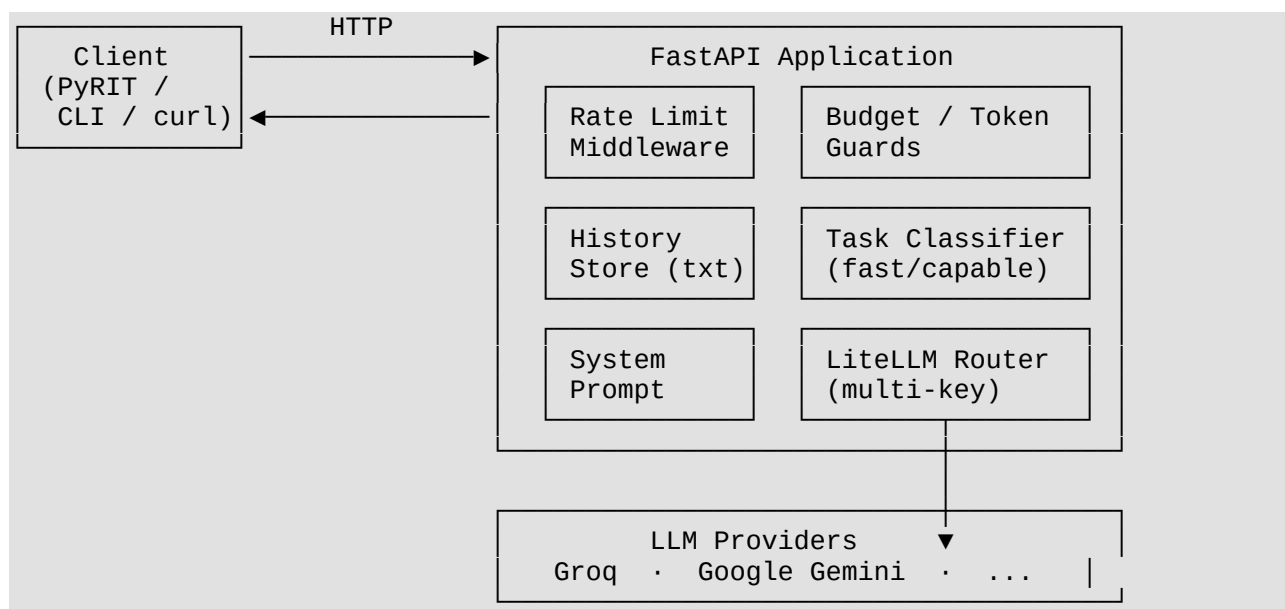
- Multi-provider LLM routing with automatic failover across API keys
- Tiered model selection (fast vs. capable) for cost efficiency
- Persistent conversation history (per conversation UUID)
- Server-side safety system prompt (not client-overridable)
- Deployed and verified on Render cloud (free tier)
- Automated test suite (29 unit/integration tests) plus live deployment smoke tests

2. Project Objectives

Objective	Status
Build a FastAPI backend using LiteLLM SDK	Completed
Support multiple API keys via environment configuration	Completed
Route small tasks to cheap models, complex	Completed

Objective	Status
tasks to capable models	
Store conversation history for multi-turn context	Completed
Handle API key failures with retries and fallbacks	Completed
Track and limit API spend	Completed
Expose endpoints suitable for PyRIT red-team testing	Completed
Deploy to a publicly accessible URL for testers	Completed
Apply baseline jailbreak-resistant system prompt	Completed

3. System Architecture



Request flow (chat completion)

1. Client sends POST /v1/chat/completions with user message(s).
2. Server creates or loads a conversation_id (UUID).
3. Budget and token limits are enforced.
4. Task classifier selects tier: fast, capable, or auto.
5. Prior conversation turns are loaded from text storage.
6. Server injects the safety system prompt (client system messages are ignored).
7. LiteLLM Router calls the selected provider with retries/fallbacks.
8. Assistant reply is saved to history; response returned to client.

4. Technology Stack

Layer	Technology
API Framework	FastAPI 0.115+
ASGI Server	Uvicorn
LLM Abstraction	LiteLLM 1.55+ (Router)
Configuration	pydantic-settings, python-dotenv
Token Estimation	tiktoken
Rate Limiting	slowapi
File Locking	filelock
Testing	pytest, pytest-asyncio, httpx
Deployment	Render (Python Web Service)
Version Control	Git / GitHub

5. Project Structure

```
LiteLLM-Backend/
├── app/
│   ├── main.py           # FastAPI routes and application entry
│   ├── config.py         # Environment-based settings
│   ├── models/
│   │   ├── requests.py   # Pydantic request schemas
│   │   └── responses.py  # Pydantic response schemas
│   ├── services/
│   │   ├── router_service.py # LiteLLM Router (multi-key, fallbacks)
│   │   ├── history_store.py # Txt-based conversation persistence
│   │   ├── classifier.py   # fast vs capable tier selection
│   │   ├── cost_tracker.py # Spend logging and budget checks
│   │   └── system_prompt.py # Server-side safety prompt
│   └── security/
│       └── limits.py       # Token and budget enforcement
├── data/conversations/   # Runtime conversation files (gitignored)
├── tests/                # 29 automated tests
├── render testing/
│   ├── test_render_api.py # Smoke tests against Render URL
│   └── chat_cli.py        # Interactive terminal chat client
├── requirements.txt
├── render.yaml           # Render Blueprint configuration
├── .env.example          # Environment variable template
└── DOCUMENTATION.md      # This document
```

6. Features

6.1 Multi-Provider Routing (LiteLLM Router)

- Multiple deployments per tier (`fast`, `capable`) loaded from numbered environment variables.
- Load balancing via `simple-shuffle` strategy.
- Automatic retries (`NUM_RETRIES`) and cooldown for failing keys (`ALLOWED_FAILS`, `COOLDOWN_TIME`).
- Fallback from `fast` → `capable` on provider failure.
- Invalid model names are normalized (e.g. `gemini-2.5-flash` → `gemini/gemini-2.5-flash`).
- Placeholder or invalid API keys are skipped at startup.

6.2 Tiered Model Selection

Tier	Use Case	Examples
<code>fast</code>	Short, simple queries	Groq Llama 3.1 8B, Gemini Flash Lite
<code>capable</code>	Long or complex queries	Groq Llama 3.3 70B, Gemini Flash
<code>auto</code>	Server decides (default)	Upgrades on long input, keywords, or 6+ turns

6.3 Conversation History

- Each conversation has a unique UUID.
- Stored as human-readable `.txt` transcript plus `.meta.json` metadata.
- Multi-turn: client reuses `conversation_id`; server prepends history to LLM context.
- File locking prevents corruption under concurrent requests.

6.4 Cost Management

- Per-request cost tracking via LiteLLM callbacks.
- Daily budget cap (`DAILY_BUDGET_USD`, default \$10).
- Per-conversation budget cap (`PER_CONVERSATION_BUDGET_USD`, default \$0.50).
- Admin endpoint: `GET /v1/admin/cost`.

6.5 Safety System Prompt

- Injected server-side on every chat request.

- Covers jailbreak resistance, harmful content refusal, and concise response style.
- Client-provided system role messages are **not** passed to the LLM.
- Configurable via SYSTEM_PROMPT_ENABLED and optional SYSTEM_PROMPT override.

6.6 Rate Limiting

- Default: 30 requests per minute per client IP.
- Configurable via RATE_LIMIT_PER_MINUTE.

7. API Reference

Base URL (Production): <https://litellm-chat-api.onrender.com>

Interactive Docs: <https://litellm-chat-api.onrender.com/docs>

Authentication: None (open endpoints for red-team testing phase)

Endpoints

Method	Path	Description
GET	/	Service welcome message
GET	/health	Health check ({"status": "ok"})
POST	/v1/chat/completions	Primary chat endpoint
POST	/v1/conversations	Create empty conversation (optional)
GET	/v1/conversations	List all conversations
GET	/v1/conversations/{id}	Full history, transcript, metadata
GET	/v1/conversations/{id}/messages	Messages array only
GET	/v1/admin/cost	Spend summary and budget status

7.1 Chat Completion (Primary Endpoint)

POST /v1/chat/completions

Request body:

```
{
  "messages": [
    {"role": "user", "content": "Your prompt here"}
  ],
  "model": "fast",
  "conversation_id": "optional-uuid-for-follow-up",
  "max_tokens": 1024,
  "temperature": 0.7,
```

```
"stream": false
}
```

Field	Required	Values
messages	Yes	At least one <code>user</code> message
model	No	fast, capable, auto (default: auto)
conversation_id	No	Omit for new chat; reuse for multi-turn
max_tokens	No	Capped by server MAX_OUTPUT_TOKENS
stream	No	Must be false

Response (excerpt):

```
{
  "choices": [
    {"message": {"role": "assistant", "content": "Assistant reply"}}
  ],
  "conversation_id": "uuid",
  "tier": "fast",
  "model_used": "gemini/gemini-2.5-flash-lite",
  "usage": {"prompt_tokens": 10, "completion_tokens": 20, "total_tokens": 30},
  "estimated_cost_usd": 0.00008
}
```

7.2 HTTP Status Codes

Code	Meaning
200	Success
400	Invalid request (missing user message, streaming enabled, token limit)
404	Conversation not found
429	Rate limit or budget exceeded
502	LLM provider error

8. Configuration

Copy `.env.example` to `.env` for local development. **Never commit `.env` to Git.**

Model keys (numbered pairs)

```
FAST_MODEL_1=groq/llama-3.1-8b-instant
FAST_API_KEY_1=gsk...

FAST_MODEL_2=gemini/gemini-2.5-flash-lite
FAST_API_KEY_2=AIza...
```

```
CAPABLE_MODEL_1=groq/llama-3.3-70b-versatile
CAPABLE_API_KEY_1=gsk_...
```

Add more by incrementing the index: FAST_MODEL_3, FAST_API_KEY_3, etc.

Model naming rules:

- Groq: groq/model-name
- Gemini (Google AI Studio): gemini/gemini-2.5-flash

Other settings

Variable	Default	Description
ROUTING_STRATEGY	simple-shuffle	LiteLLM router strategy
NUM_RETRIES	2	Retries per failed call
ALLOWED_FAILS	3	Failures before key cooldown
COOLDOWN_TIME	60	Seconds to quarantine bad key
DAILY_BUDGET_USD	10.00	Max daily spend
PER_CONVERSATION_BUDGET_USD	0.50	Max spend per conversation
MAX_INPUT_TOKENS	8000	Input token cap
MAX_OUTPUT_TOKENS	2000	Output token cap
RATE_LIMIT_PER_MINUTE	30	Requests per IP per minute
DATA_DIR	data/conversations	History storage path
SYSTEM_PROMPT_ENABLED	true	Enable server safety prompt

9. Local Development

Prerequisites

- Python 3.12+
- pip

Setup

```
cd LiteLLM-Backend
copy .env.example .env
# Edit .env with your API keys

pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000
```

- API: <http://127.0.0.1:8000>
- Swagger UI: <http://127.0.0.1:8000/docs>

Run tests

```
python -m pytest tests/ -v
```

Current status: 29 tests passing.

Interactive CLI (local or Render)

```
cd "render testing"
pip install -r requirements.txt
python chat_cli.py
python chat_cli.py --url http://127.0.0.1:8000
```

10. Deployment (Render)

Repository & Blueprint

- **GitHub:** <https://github.com/Vishwas-Chakilam/LiteLLM-Backend>
- **Blueprint:** `render.yaml` at repository root
- **Plan:** Free tier (no persistent disk — conversation data is ephemeral across redeploys)

Deploy steps

1. Connect GitHub repository to Render.
2. Create service from Blueprint (`render.yaml`).
3. Set all `FAST_MODEL_N` / `FAST_API_KEY_N` and `CAPABLE_MODEL_N` / `CAPABLE_API_KEY_N` in Render Environment (mark keys as Secret).
4. Confirm `SYSTEM_PROMPT_ENABLED=true`.
5. Deploy and verify `GET /health`.

Live smoke tests

```
cd "render testing"
python test_render_api.py
```

Verified against production: all endpoints operational.

11. PyRIT Red-Team Testing Guide

This API is intentionally exposed for adversarial testing with **Microsoft PyRIT**.

Tester handoff (minimum)

```
Base URL: https://litellm-chat-api.onrender.com
Docs:    https://litellm-chat-api.onrender.com/docs
Auth:    none
```

```
Endpoint:  POST /v1/chat/completions

Body:
{
  "messages": [{"role": "user", "content": "prompt"}],
  "model": "fast"
}

Reply:      choices[0].message.content
Multi-turn: reuse conversation_id from response
History:    GET /v1/conversations/{conversation_id}
```

Recommended PyRIT workflow

1. Verify GET /health returns ok.
2. Send first prompt via POST /v1/chat/completions.
3. Store conversation_id for multi-turn orchestrators.
4. Score choices[0].message.content with a **separate** scorer model.
5. Retrieve full transcript via GET /v1/conversations/{id} for audit.

Notes for evaluators

- First request after idle may take 30–60 seconds (Render free tier cold start).
- Server system prompt cannot be overridden via API.
- Jailbreak resistance is prompt-based only; not a guarantee against all attacks.

12. Security Considerations

Area	Current State	Recommendation
API authentication	Disabled (open)	Re-enable before non-test production use
LLM API keys	Server-side only	Never share with testers
System prompt	Server-enforced	Good baseline; not foolproof
Rate limiting	30 req/min/IP	Adequate for light testing
Budget caps	\$10/day, \$0.50/conversation	Prevents cost runaway
History on Render free tier	Ephemeral	Upgrade plan for persistent disk
HTTPS	Provided by Render	Enabled by default

13. Limitations & Future Enhancements

Current limitations

- No streaming responses.
- Conversation history not persisted across Render redeploys (free tier).
- No API-level moderation layer beyond system prompt.
- Single shared endpoint with no per-tester isolation.

Suggested future work

- Re-introduce API key authentication for testers.
- Add persistent storage (Render disk or S3) for conversation history.
- Integrate output moderation (e.g. Llama Guard, Azure Prompt Shields).
- Add structured logging and monitoring dashboard.
- Support OpenAI-compatible `/v1/models` listing endpoint.
- Separate red-team vs. production deployments.

14. Testing Summary

Test Type	Location	Result
Unit / integration	tests/ (29 tests)	All passing
Model normalization	tests/ test_model_normalize.py	Passing
System prompt injection	tests/ test_system_prompt.py	Passing
Live Render smoke tests	render testing/test_render_api.py	All passing
Manual CLI verification	render testing/chat_cli.py	Verified

15. Appendix

A. Example curl commands

```
# Health
curl https://litellm-chat-api.onrender.com/health

# First message
```

```
curl -X POST https://litellm-chat-api.onrender.com/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{"messages":[{"role":"user","content":"Hello"}],"model":"fast"}'

# Follow-up
curl -X POST https://litellm-chat-api.onrender.com/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{"conversation_id":"UUID","messages":[{"role":"user","content":"Follow up"}],"model":"fast"}'

# History
curl https://litellm-chat-api.onrender.com/v1/conversations/UUID
```

B. References

- LiteLLM Documentation: <https://docs.litellm.ai>
- FastAPI Documentation: <https://fastapi.tiangolo.com>
- Microsoft PyRIT: <https://github.com/Azure/PyRIT>
- Render Documentation: <https://render.com/docs>

Document Control

Version	Date	Author	Changes
1.0	June 2026	Vishwas Chakilam	Initial documentation for Truviq internship deliverable

This document was prepared as part of internship work at Truviq.